



LES PROCESSUS : NOTION DE BASE

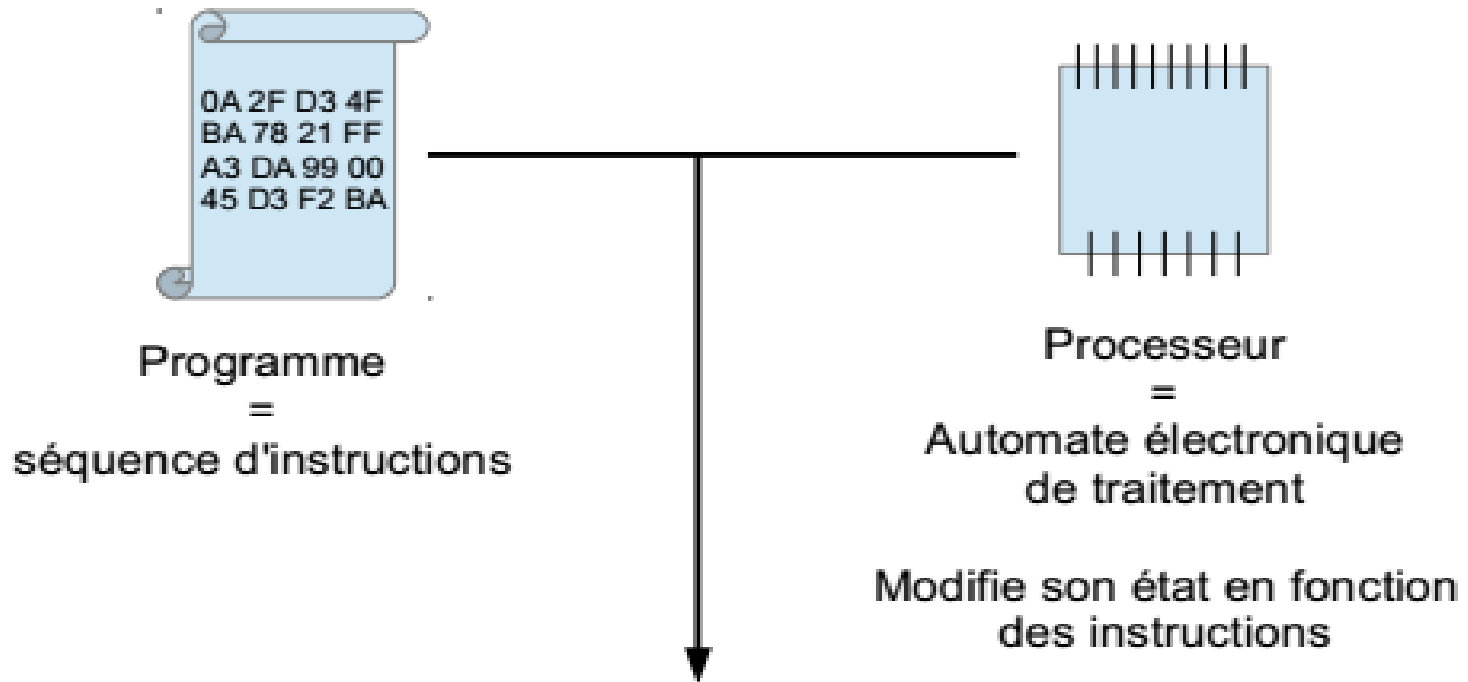
INTRODUCTION

2

- Un programme en exécution dans l'ordinateur est appelé
.....
- Un système d'exploitation / **Operating**
systeme (S.E. ou O.S. en anglais) qui traite plusieurs processus simultanément fait du multitâches.
- Un ordinateur à un seul processeur traite **une tache**.....
à la fois, s'interrompt et passe à la suivante. La transition est si rapide, donne l'illusion **d'effectuer un**
Traitement simultané.....

Définition (1)

3



Un processus = un programme exécuté par un processeur.....

Définition (2)

4

- Programme $\neq$ processus
 - Programme = ensemble d'instructions

.....Execution d'un programme par l'UC = processus.....

.....=> Un programme peut générer plusieurs processus.....
- Exemples de processus :
 - Logiciel de traitement de texte en cours d'utilisation
 - Compilation d'un code source
 - Tâche système (envoi de données vers l'imprimante)

Définition (3)

5

- **Un processus** est une entité **dynamique** qui matérialise un **programme** **en cours d'exécution** avec ses propres ressources **physique** (mémoire, processeur, entrée/sortie, ...) et **logiques** (données, variables, fichiers, ...)
- **Ne pas confondre un processus avec programme**
 - Un programme est une suite d'instructions ; c'est du texte, un code **statique**
 - Le processus est un concept dynamique, il représente **le déroulement** d'une tâche faisant partie d'une application ou un programme système quelconque.
- **Un processeur exécute un seul programme à la fois => processus**

Notion de processus (1)

6

- Un processus peut appeler d'autres processus que l'on appelle **sous-processus**.
- On appelle le processus créateur, **père**....., et les processus créés, **fil**s.....
- Les processus peuvent donc se structurer sous la forme **arborescence**.....
- Au lancement du système, il n'existe qu'un seul **processus**....., qui est **ancêtre**.....de tous les autres.

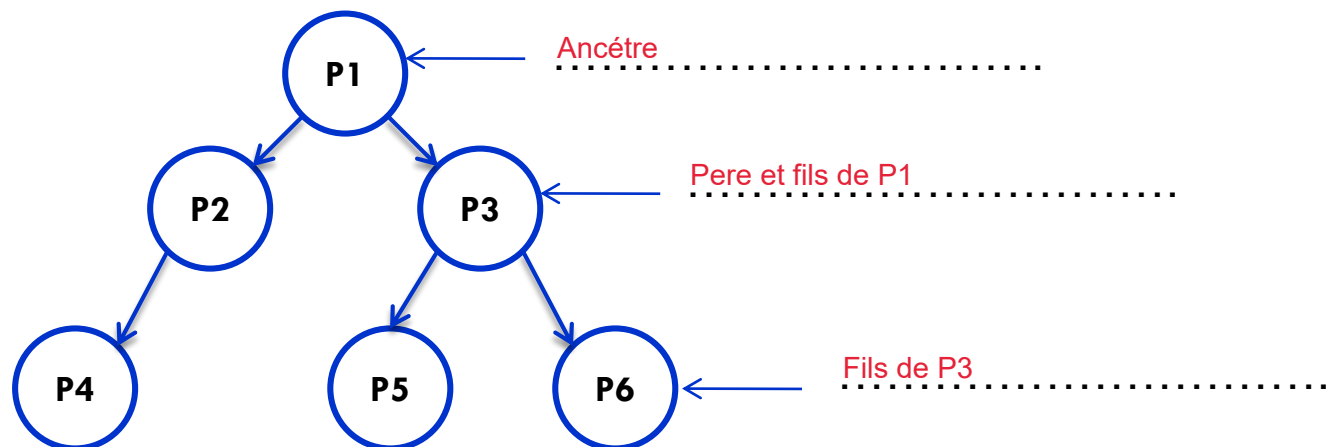


Figure1: Le hiérarchie des processus

Notion de processus (2)

7

- Chaque S.E. crée et maintient un certain nombre d'informations sur les processus en cours d'utilisation dans l'ordinateur.
- Dans cette table des processus, on y retrouve presque toujours :
 - .Identificateur unique du processus.....;
 -L'état du processus.....;
 -le niveau de priorité du processus.....;
 -table de pointeurs sur les ressources utilisées.....;

Notion de processus (3)

8

- Le passage d'un processus à un autre est réalisé par unordonnanceur..... du système, lequel est activé par desinterruptions..... : d'horloge, de disque, de terminaux.
- La différence entre un processus et un programme est mince : le processus possède le programme mais égalementl'état courant..... de celui-ci dans la mémoire de l'ordinateur.

Notion de processus (4)

9

- Le S.E. se sert d'unalgorithme d'ordonnancement..... qui détermine à quel moment il faut suspendre un processus et à quel moment en repartir un autre.
- Pour les systèmes multitâches, l'initialisation démarre parla création du processus père..... qui se trouve à latête de l'arborance..... de processus.
- Les commandes provenant alors des autres programmes créent d'autres processus qui se greffent à l'arbre unique des processus du système.

Notion de processus (5)

10

Notion de processeur

11

- Le **processeur** (ou l'unité centrale de traitement (UCT), UCT, en anglais Central processing Unit, CPU) est le composant de votre ordinateur qui exécute les instructions qui lui sont données par votre système d'exploitation (Windows).
- Quand vous lancez un logiciel, regardez une vidéo haute définition ou lorsque vous jouez à un jeu vidéo, vous faites travailler en priorité le processeur.....
- Pour répondre à vos demandes les plus exigeantes, le **processeur** peut être doté des plusieurs Coeurs.....

Notion de processeur (1)

Registre de processeur

12

- Un **registre** est un emplacement de**mémoire interne à**
.....**un processeur**.....
- Les registres se situent au sommet de la hiérarchie mémoire : il s'agit de la**mémoire la plus rapide dans ordinateur**
....., mais dont le coût de fabrication est le plus élevé car la place dans un microprocesseur est limitée.
- Leur capacité dépasse donc rarement**quelques mégas octets**
.....

Notion de processeur (2)

13

- Un **processeur standard** possède **un coeur**..... (on dit qu'il est *single-core*).
- Un **processeur avec 1 seul cœur** ne peut traiter qu' **une seule instruction à la fois**..... (des instructions peuvent être traitées ensemble mais toujours une par une).
- Une instruction est une tâche qu'on demande au processeur d'exécuter : convertir une vidéo, compresser des fichiers volumineux, etc.

Notion de processeur (3)

14

- Un **processeur multi-cœur** est composé de deux ou plusieurs**Cœurs Indépendants**..., chacun étant capable de traiter des instructions individuellement.
- Un **processeur dual-core** contient deux cœurs, un **processeur quad-core** quatre cœurs, un **processeur hexa-core** six cœurs...



Notion de processeur (4)

15

- Un **processeur multi-cœur** permet à l'utilisateur d'..... sans subir de ralentissements.
- Autrement dit, les cœurs sont utiles si vous utilisez plusieurs logiciels
à la fois.....
- Quand un programme est en cours d'exécution (logiciel de montage vidéo ou de retouche photo par exemple), vous pouvez utiliser pour utiliser votre navigateur internet ou écrire un document :pc:

Notion de processeur (5)

16

- Avoir **plusieurs cœurs** est aussi utile lorsque vous utilisez un logiciel qui peut **utiliser plus d'un cœur**.
- En effet, la majorité des programmes est conçue pour n'utiliser qu'un seul et unique cœur.
- Un logiciel qui est **compatible avec le multicœur** fonctionne lui beaucoup plus rapidement puisqu'il peut exécuter plusieurs instructions en même temps. C'est le cas notamment des logiciels de retouche photo ou les jeux vidéo.

Notion de processeur (6)

17

- ❑ Si vous n'utilisez votre ordinateur que pour consulter vos mails ou regarder des vidéos, vous n'avez pas vraiment besoin d'un ..processeur.. avec ..beaucoup de coeurs



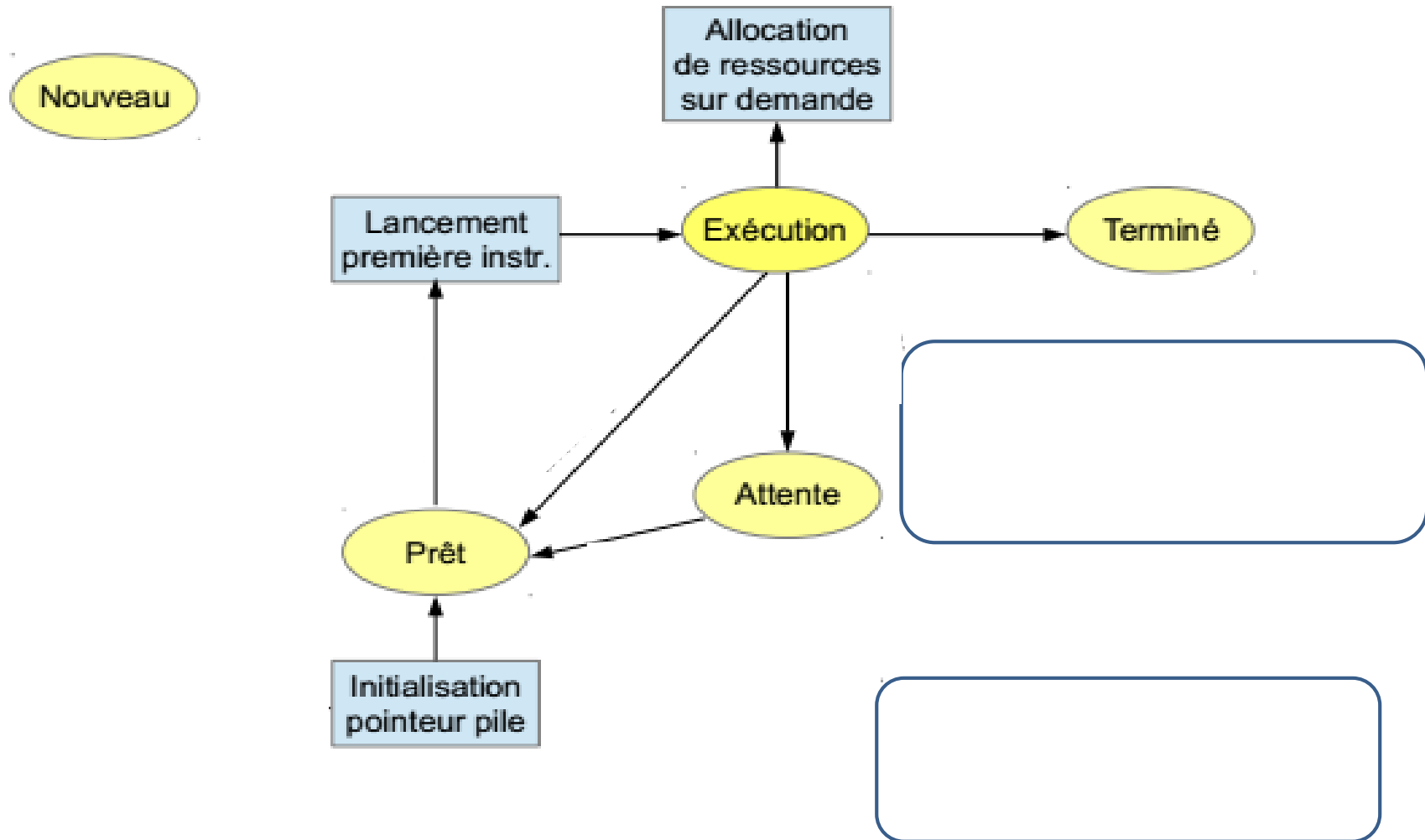
Notion de processeur (7)

18

- Le est certes le premier indicateur de la mais il ne faut pas se focaliser uniquement sur cette donnée.
- D'autres caractéristiques, notamment chez les **processeurs Intel Core** :, la fréquence, la mémoire
....., les technologies utilisées

Cycle de vie du processus

19



Structure d'un proc. en mémoire

20

□ Environnement

- ▣ Variables héritées du parent (ex : PATH=/usr/bin)

□ Tas

- ▣ Données globales statiques + variables allouées dynamiquement
→ partagé par tous les threads du processus

□ Pile

- ▣ Variables temporaires du programme (blocs C)
- ▣ Contexte (ex : compteur d'instruction)

□ Code

- ▣ Recopie du programme machine : fixe et protégé en mémoire
- ▣ C'est le segment « exécutable » du processus

Relation entre processus (1)

21

- L'exécution d'un processus dure un certain temps, avec **Début**..... et (parfois) **une fois**...
- Un processus peut être démarré par **Un utilisateur**..... par l'intermédiaire d'un périphérique ou bien par **un autre processus**..
- Les applications utilisateur (*un éditeur de texte, un navigateur web, un lecteur multimédia, un jeu vidéo*) sont **Des ensembles de processus**.....

Relation entre processus (2)

22

edition du lien : permet de passer
d'une adresse symbolique à une
adresse logique.

Bibliothèque du systeme : Arbitrage et autorisation

Bibliothèque du standards : arbstration et augmentation

Quelle sonts les fonctions responsables à la création processus sous windows et sous
linux?

Linux==> fork() + exec() (si on veux le fils)

Windows => createprocess ()

Relation entre processus (3)

23

- Un processus ne pouvant conceptuellement être créé qu'au moyen **d'un appel système**, c'est à dire par un autre processus ayant réalisé **.....appel.....**
- La fonction **Createprocess** fait partie des appels système standard de **Windows**.....
- La fonction **Fork**.....fait partie des appels système standard **d'UNIX**..... Cette fonction permet à un processus de donner naissance à un nouveau processus qui est sa copie conforme.
- Par exemple en vue de réaliser un second traitement parallèlement au premier.

Création de processus

Sous Windows

24

- Appel système **CreateProcess** : crée un processus dont les caractéristiques sont données en paramètres de l'appel système.
- Exemple :

```
STARTUPINFO si; // à renseigner  
PROCESS_INFORMATION pi; // utilisé en sortie  
bool succes = CreateProcess( 0, "mon_prog.exe", 0, 0,  
FALSE, 0, 0, 0, &si, &pi);
```

Création de processus

Sous UNIX (1)

25

- Appel système **fork** : crée un processus dont les caractéristiques sont *identiques* à celles du processus courant (seuls le PID et le PPID changent).

→ retourne une valeur différente dans le processus père et le processus fils.

- Exemple :

```
int pid_fils = fork();  
if (pid_fils == 0) return fonction_fils();  
    else return fonction_père();
```

Création de processus

Sous UNIX (2)

26

- Si on souhaite que le fils exécute un autre programme que le père, on utilise une variante de **exec** (qui **remplace** le programme courant par un autre programme).
- Exemple :

```
int pid_fils = fork();  
if (pid_fils == 0) execlp("mon_prog", "mon_prog");  
// seul le père exécute la suite
```

Création de Plusieurs processus

Sous UNIX (3)

27

```
#include <stdio.h>

Void main () {
int code_retour ;
printf ("Debut du test fork ()\n");
code_retour = fork ();
switch (code_retour ) {
case -1 :
printf ("Pbm lors de la creation du processus\n"); break;
case 0 :
printf ("Je suis le processus fils \n"); break;
Default :
printf ("Je suis le processus père\n");
printf ("Je viens de créer le processus fils dont le pid est %d \n",code_retour); break; }
printf ("code_retour %d\n", code_retour);
printf ("Fin du test fork ()\n"); }
```

Création de Plusieurs processus

Sous UNIX (4)

28

Code C sous Unix

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
void main() {
    pid_t fils;
    printf("Starting ...\n");
    fils = fork();
    if (fils==-1) fprintf(stderr, "fork failed");
    else {
        if (fils==0) /* c'est le fils */ {
            printf("%d: fils de %d \n", getpid(), getppid());
            sleep(5);
            printf("%d: termine \n",getpid());
            exit(5);
        } else /* c'est le pere */ {
            printf("%d: pere de %d \n", getpid(), fils);
            int r; waitpid(fils, &r, 0); /* attente du fils */
            printf("%d: fils %d sort (code %d)\n",getpid(),fils,r);
        }
    }
}
```

Création de processus (1)

Comparaison

29

- Le modèle de **Windows** est plus simple à appréhender: création d'un nouveau processus « **à partir du zero**..... ».
- Le modèle d'**UNIX** est plus .portable et évolutif..... : il reste valable quelles que soient les informations nécessaires à la création d'un processus.
 - ▣ Chaque processus produit, lorsqu'il se termine, une valeur entière (valeur retournée par la fonction main).
 - ▣ Cette valeur doit être récupérée par le processus parent (appel système**wait**.....).
 - ▣ Tant que cela n'a pas été fait, les ressources du processus enfant ne peuvent pas être libérées ; le processus est dans un état particulier appelé « **Zombie**..... » (entre Élu et terminé).

Création de processus (2)

30

Trois événements qui peuvent provoquer la création d'un processus:

- **Initialisation du système : processus système**
 - Processus de premier plan : interaction avec l' utilisateur.
 - Processus d'arrière plan : processus non associé à un utilisateur.
- **Exécution d'un appel système : fork (Linux) / CreateProcess (Windows)**
 - ▣ Techniquement, c'est le seul moyen pour lancer un Processus.
- **Requête utilisateur**
 - ▣ Exécution de commande sur un interpréteur de commandes.
 - ▣ Double clicks sur une icône.

Filiation des processus (1)

31

- Le créateur d'un nouveau processus est appelé le ...**père**... et le nouveau processus, le **....fils**.....
- Tous les attributs système du père (par exemple les droits sur le système de fichier) sont transmis au fils.
- Au démarrage d'un système Unix, un seul processus existe qui est **....l'ancêtre**..... commun à tous de numéro 1 (processus **....init**....).
- Tous les autres processus, qui peuvent exister au cours de la vie du système, descendent de ce premier processus via **....des appels système**.....
....fork successifs
.....

Filiation des processus (2)

32

- L'appel système `fork` fournit une valeur résultat qui est un entier.
- Pour identifier le père du fils nouvellement créé, il suffit de regarder la valeur de retour du `fork()` que l'on appelle plus communément le **PID** (*identifiant de processus*).
- Si le **PID** vaut `0`, alors il s'agit du **fils**, sinon c'est le **père**.
- Afin d'obtenir le numéro du processus, il suffit de faire l'appel système `getpid()`, ou `getppid()` pour obtenir le numéro du père.

Gestion de processus (Services POSIX)

33

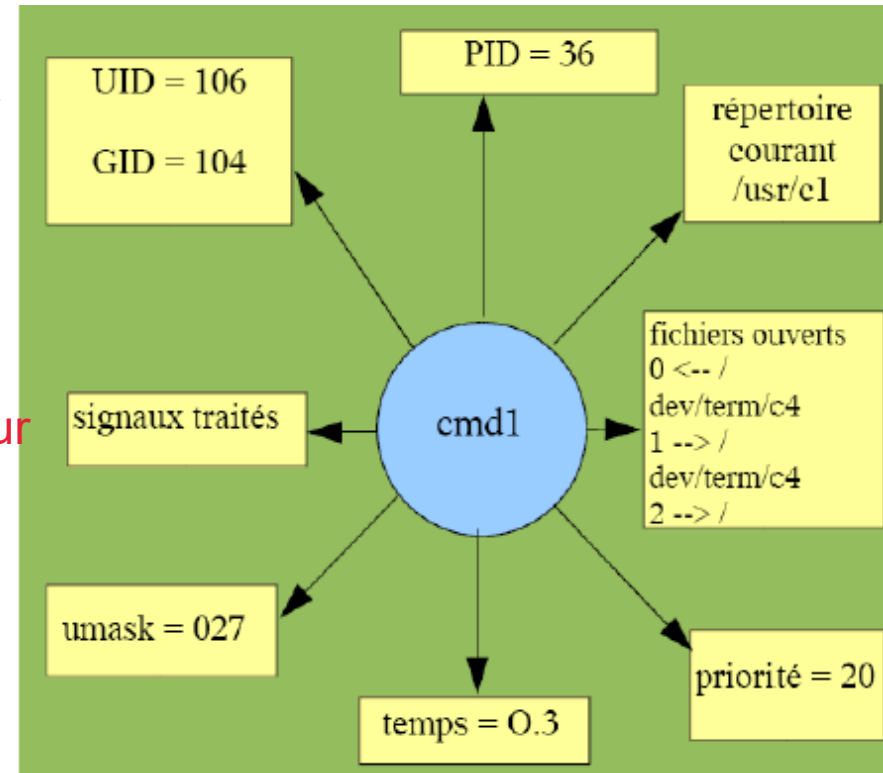
- Cette norme propose la spécification d'un ensemble de fonctions permettant de solliciter les services de base d'un système d'exploitation.
- La norme Posix définit un nombre relativement petit d'appels système pour la gestion de processus
 - `pid_t fork()` :Création du processus fils.....
 - `int execlp()` : ...Exécuter un programme (code) différent.....
 - `pid_t wait()` :attendre la terminaison d'un processus.....
 - `void exit()` :Finir l'exécution d'un processus.....
 - `pid_t getpid()` :Retourne l'identifiant du processus.....
 - `pid_t getppid()` :retourne l'identifiant du processus père.....

Paramètre d'un processus

34

□ Dès sa création, un processus reçoit les paramètres suivants:

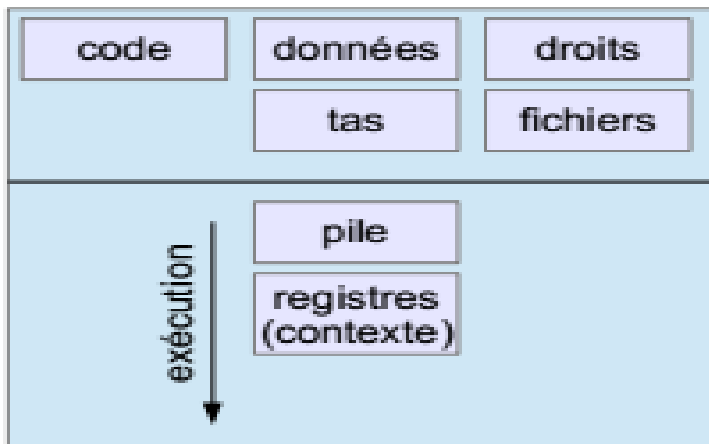
- **PID:** ...Identificateur de processur...;
- **PPID:** ...Identifcateur du processus père...;
- **UID:** ...Identifcateur de l'utilisateur qui ...
...a lancé le programme.....;
- **GID:** ...Identificateur du groupe d'utilisateur
qui a lancé le programme.....;



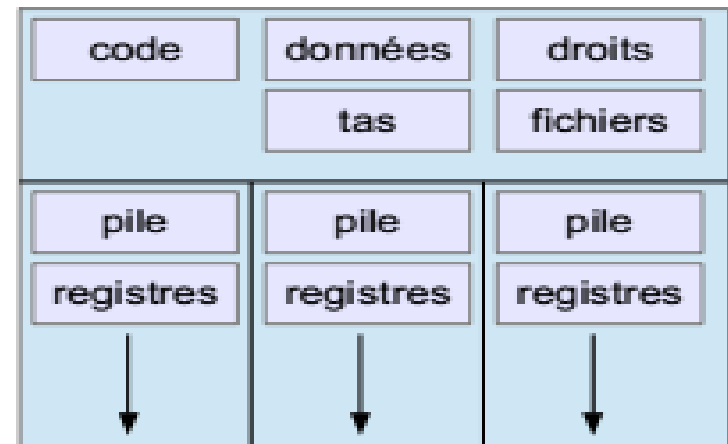
Thread : définition

35

- Une thread est l'unité de base du processus



Processus mono-thread



Processus multi-thread

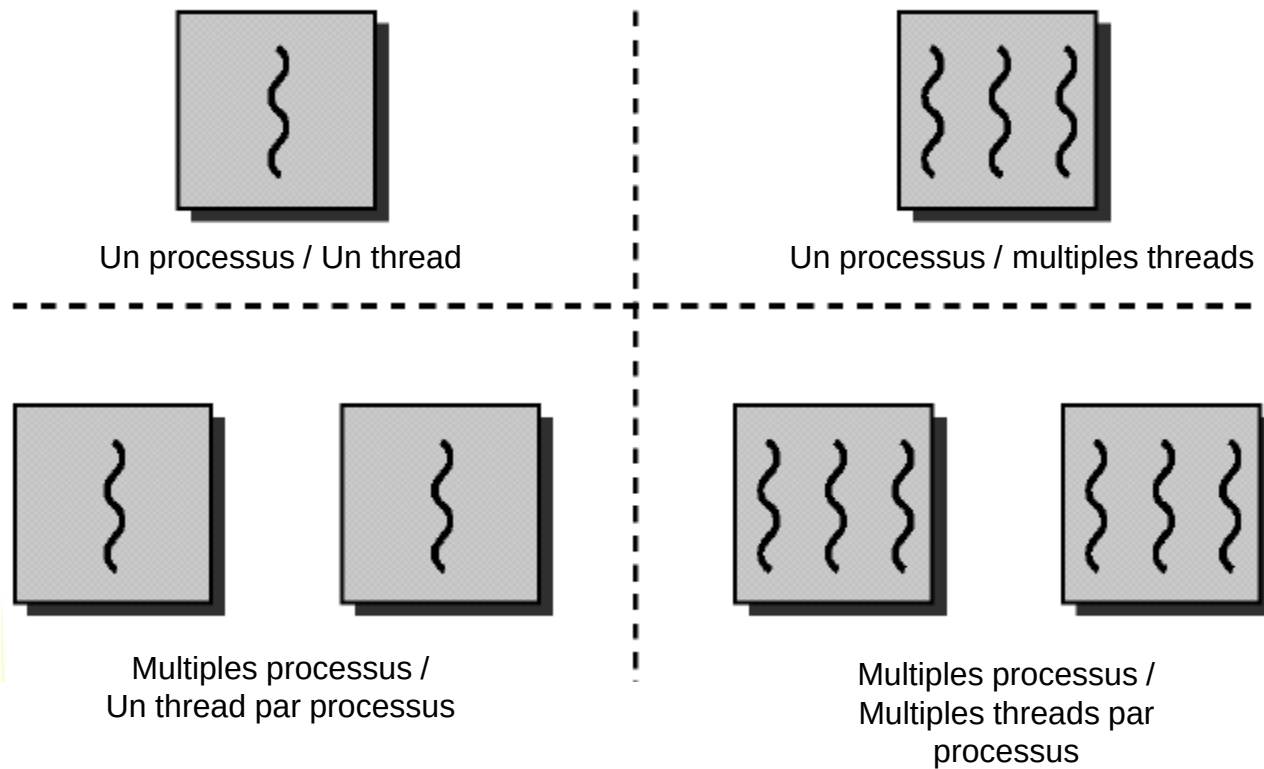
Processus vs Thread (1)

36

- ❑ Chaque processus peut fonctionner comme le système d'exploitation en lançant des Ces sous-tâches sont nommées « ».
- ❑ Le processus est vu comme étant un ensemble de (code exécutable, segments de données, de fichiers, de périphériques, etc.) que les partagent.
- ❑ Le a été introduit dans les systèmes afin de permettre la mise en œuvre du parallélisme à l'intérieur des processus lourds.

Processus vs Thread (2)

37



Thread : exemple

38

□ Le processus MS-Word

▣ Il peut impliquer plusieurs **threads**:

■

■

■

■

■ Etc

▣ Ces threads partagent tous le même fichier .doc et autres données

→ processus

Pourquoi Les Threads ? (1)

39

□

- ▣ Un processus multi-threadé peut continuer à s'exécuter pendant qu'une de ses threads est bloquée
- ▣ Ex : navigateur web → chargement des images

□

- ▣ Ex: le code n'est pas recopié N fois

□

- ▣ Allocation mémoire, Exemple sous Solaris, la création d'un processus est 30 fois plus lente que celle d'un Thread

Pourquoi Les Threads ? (2)

40

-
.....
 - **Exemple:** Dans un logiciel de traitement de texte, on utilise des threads pour les activités différentes (enregistrement, vérification orthographe, formatage à l'écran,...)
 - Changer d'un thread à un autre dans le même processus est bien plus simple, implique sauvegarder seulement (les registres de l'UCT, la pile du thread, les vars locales du thread).
- **Communication facile entre les Threads.**
-
 - Sur les architectures multiprocesseurs

États d'un processus (1)

41

- Le système d'exploitation est chargé d'allouer les ressources (mémoires, temps processeur, entrées/sorties) nécessaires aux processus et d'assurer que le fonctionnement d'un processus n'interfère pas avec celui des autres.
- Les processus passent par différents. Les d'un processus sont généralement représentées par un **diagramme d'état**.

États d'un processus (2)

42

- En exécution, un processus change d'état:
 -: processus en train d'être créé
 -: processus en exécution
 -: processus en attente d'un évènement
 -: processus en attente du processeur
 -: processus exécuté et terminé

États d'un processus (3)

États standards

43

- Ces états existent dans la plupart des systèmes d'exploitation.

États d'un processus (4)

États standards

44

- **Initialisation** (*en anglais, created ou new*) :
..... Il y attend que
....., ce qui peut prendre plus ou moins longtemps.
- **Prêt ou En attente** (*en anglais, ready ou runnable*) : le processus
a été chargé
.....

Il peut y avoir beaucoup de processus en attente car, sur un ordinateur équipé d'un seul processeur, les processus doivent passer un par un.

Les processus disponibles sont rangés dans une file. Cette file d'attente (.....) est gérée par l'ordonnanceur.

États d'un processus (5)

États standards

45

- **Élu ou Exécution** (*en anglais, running*) : Le processus
.....
 - **Endormi ou Bloqué** (*en anglais, blocked ou waiting*) : Le processus (la fin d'une opération d'entrée/sortie, un signal, ...).
 - **Terminé** (*en anglais, terminated*) : Le processus est, c'est-à-dire soit le résultat est connu, soit le programme a été forcé de s'arrêter.
- ➡ États que peut prendre un processus sous la forme d'un diagramme états-transitions où les flèches sont les transitions autorisées.

États d'un processus (6)

États particuliers

46

- Selon les systèmes d'exploitation, ces différents états peuvent aussi être possibles :
- Exemple: SE Linux.

États d'un processus (7)

États particuliers

47

- **Zombie** : Si un processus
.....
.....
 - **Swappé**: Lorsqu'un processus
..... Un
processus swappé peut être dans un état
 - **Préempté** : L'ordonnanceur
.....Par exemple, un processus qui consomme
trop de temps CPU finira par être
- Un ordonnanceur préemptif utilise aussi l'indice de priorité
(nice) pour décider le processus qui sera préempté.

États d'un processus (8)

États particuliers

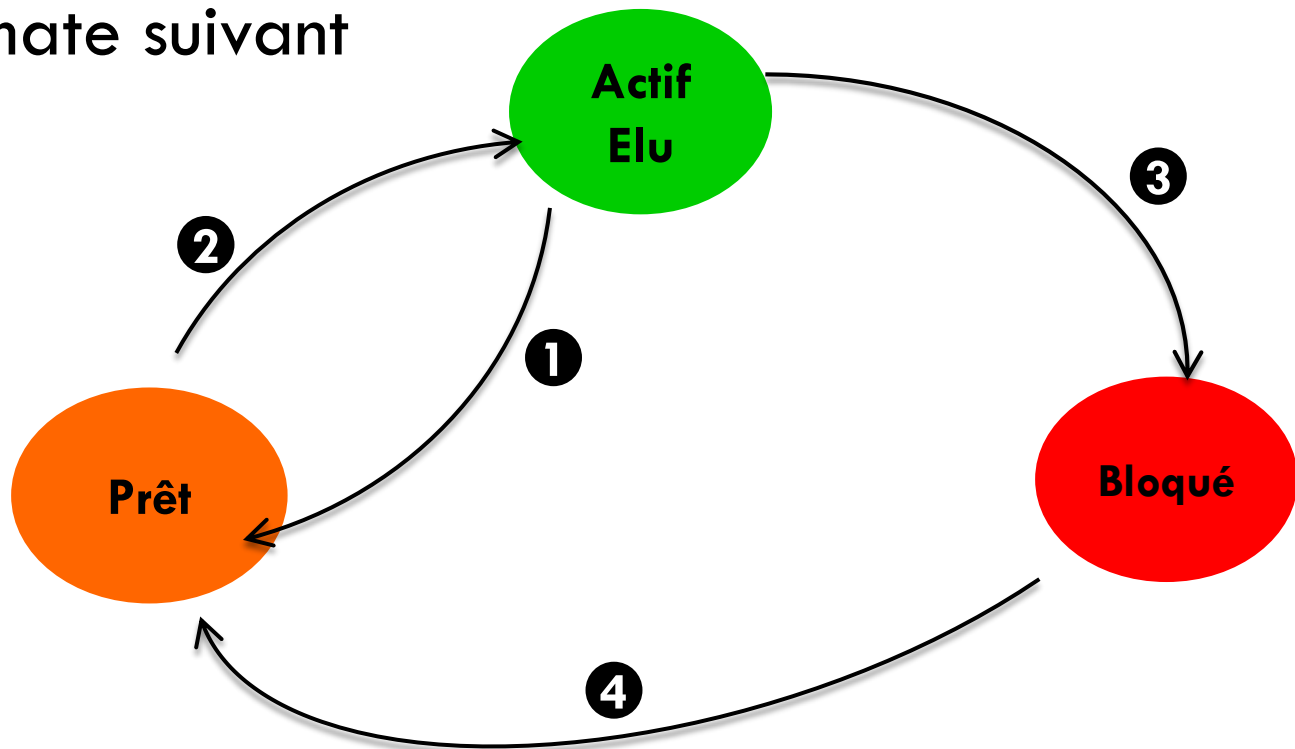
48

- **Exécution en espace utilisateur** : L'exécution a lieu dans un espace**limité**..... : seules certaines instructions sont disponibles.
 - **Exécution en espace noyau** : Par opposition au mode utilisateur, l'exécution du processus**n'est pas****limité**.....Par exemple, un processus dans cet état peut aller lire dans la mémoire d'un autre.
- ➡ Diagramme état-transition plus complexe suivi par les processus des systèmes d'exploitation modernes.

Transitions d'un processus (1)

49

- Les transitions entre ces états sont matérialisées par l'automate suivant



- La signification des quatre transitions légales est la suivante :

Transitions d'un processus (2)

50

- 1. Le processus
.....attribué.....
- 2. L'Ordonnanceur, appelé de façon asynchrone par interruption lorsque le temps imparti est écoulé,
.....élit un processus parmi les processus prêts.....
- 3. Le processus
.....externe. (décompte d'horloge, attente de données, ...).
- 4. L'événement attendu par le processus se produit. C'est le SE qui gère son traitement en interrompant temporairement le déroulement du processus actuellement élu pour traiter les données reçues, et
.....processus en attente de l'état bloqué à l'état prêt.....

Bloc de contrôle de processus (1)

51

- En multiprogrammation, un processus s'exécute **sur** **l'unité centrale de traitement (UCT)** (**le processeur**)
de façon intermittente (irrégulière).
- **Chaque fois qu'un processus reprend l'UCT (transition prêt → exécution) il doit la reprendre dans la même situation où il l'a laissée;**
- Donc au moment où un processus sort de l'état exécution il est nécessaire de **Sauvegarder ses**
..... **informations essentielles**, qu'il faudra récupérer quand il retourne à cet état.

Bloc de contrôle de processus (2)

52

- Chaque processus est représenté dans le SE par un **PCB (process control block)**.

- **PCB (Process Control Bloc)**

est une fiche signalétique liée
au processus permettant au
système d'exploitation de

disposer des informations liées

.....
à ce processus
.....

Bloc de contrôle de processus (3)

53

- **PCB:** contient plusieurs informations concernant un processus spécifique, comme par exemple:
 - ▣ L'état du processus
 - ▣ compteur ordinaire: indique l'adresse de l'instruction suivante devant être exécutée par ce processus.
 - ▣ Informations sur le scheduling de la CPU: information concernant la priorité du processus
 - ▣ Informations sur la gestion de la mémoire valeurs des registres base et limite, des tables de pages ou des tables de segments.
 - ▣ Information sur l'état des E/S: liste des périphériques E/S allouées à ce processus, une liste des fichiers ouverts, etc.

Commutation de contexte (1)

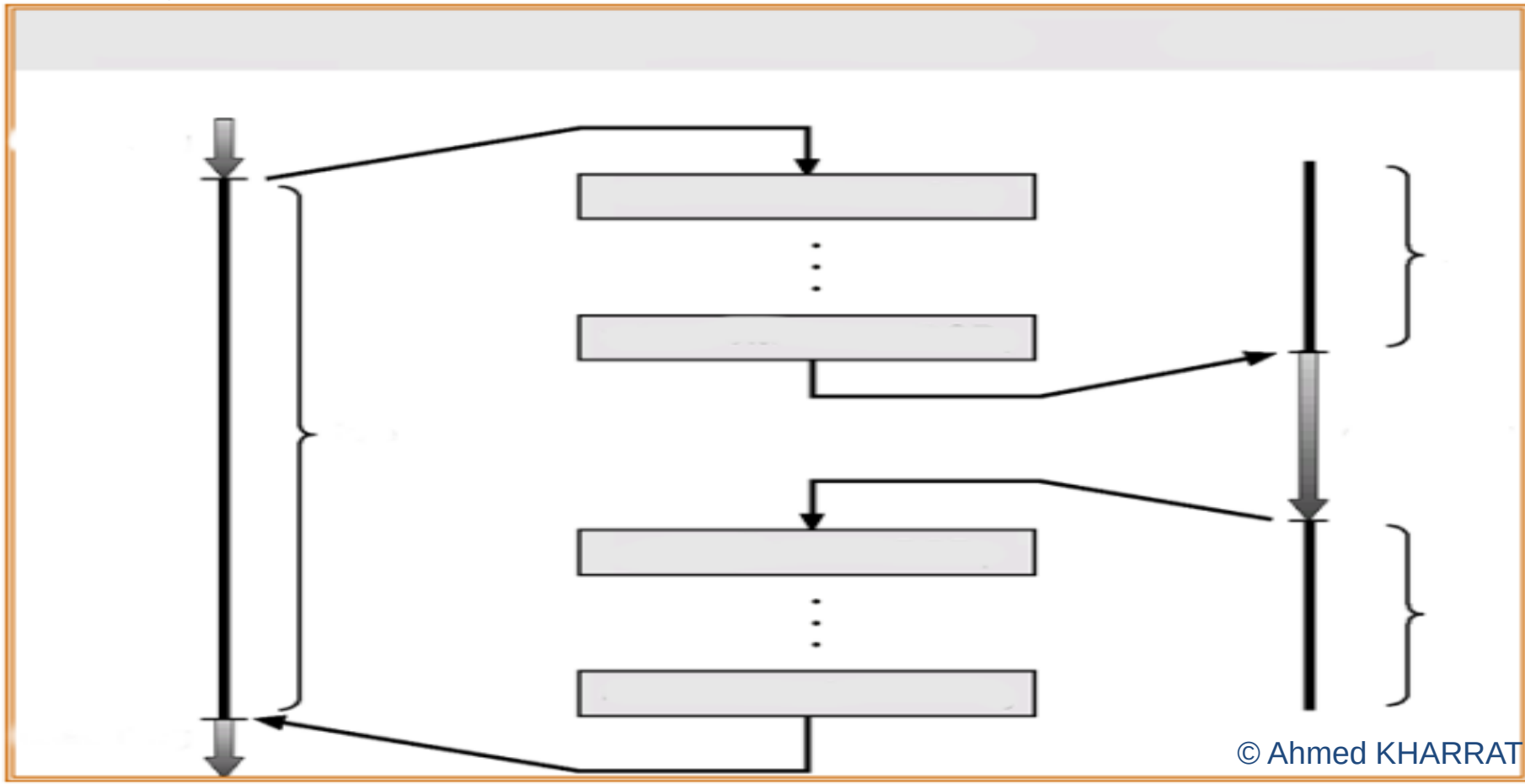
54

- Le basculement d'un processus à un autre est g  rer par le SE.....
- Lorsqu'un processus obtient le contr  le de l'UCT, le syst  me doit stocker les informations li  es au processus qui vient de quitter l'  tat   lu dans son PCB..... et r  initialiser par celles li  es au processus qui obtient le contr  le    l'UCT.
- Les informations    changer constituent Le contexte d'ex  cution d'un processus.....d'un processus.
- La r  initialisation de ces valeurs est appel   commutation de contexte.....

Commutation de contexte (2)

55

- La commutation de contexte peut être illustrée par la figure suivante :



Prochain cours: **L' Ordonnancement des processus**

Merci

Des questions?